

Name: Diya Ravindra Bhansali

PRN: 125B1F271

Class & Div: SY BTECH IT- A4

Source code and Output:

```
/*
```

```
Assignment No: 03
```

```
Assignment Title: Hospital Patient Record System Using Singly Linked List
```

```
Name: Diya Ravindra Bhansali
```

```
PRN:125B1F271
```

```
Class & Div : SY BTech IT A4
```

```
*/
```

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class Node
```

```
{
```

```
public:
```

```
    static int cnt;
```

```
    int patient_id;
```

```
    string name;
```

```
    int age;
```

```
    string condition;
```

```
    Node *next;
```

```
    Node()
```

```
    {
```

```
        next = NULL;
```

```
    }
```

```
    void get_info()
```

```
    {
```

```
        cnt++;
```

```
        patient_id = cnt;
```

```
        cout << "\nEnter Patient Name: ";
```

```
        cin >> name;
```

```
        cout << "Enter Age: ";
```

```
        cin >> age;
```

```
        cout << "Enter Patient Condition: ";
```

```
        cin >> condition;
```

```
    }
```

```

void displayInfo()
{
    cout << "\nID: " << patient_id;
    cout << "\nName: " << name;
    cout << "\nAge: " << age;
    cout << "\nCondition: " << condition << endl;
}
};

int Node::cnt = 0;
Node *head = NULL;
Node *tail = NULL;

void add_patient()
{
    Node *newPatient = new Node();
    newPatient->get_info();
    if (head == NULL)
    {
        head = newPatient;
        tail = newPatient;
    }
    else
    {
        tail->next = newPatient;
        tail = newPatient;
    }
    cout << "\nPatient added successfully!\n";
}

void display()
{
    Node *curr = head;
    cout << "\n\n**** Patient Records ****\n";
    if (head == NULL)
    {
        cout << "No patient records available!\n";
        return;
    }
    while (curr != NULL)
    {

```

```

        cout << curr->patient_id << ") "<< curr->name << " -> ";
        curr = curr->next;
    }
    cout << "NULL\n";
}

void remove_patient()
{
    if (head == NULL)
    {
        cout << "\nPatient records are empty!\n";
        return;
    }
    string sname;
    cout << "\nEnter the patient name to be removed: ";
    cin >> sname;
    if (head->name == sname)
    {
        Node *temp = head;
        head = head->next;
        if (head == NULL)
        {
            tail = NULL;
        }
        delete temp;
        cout << "\nPatient removed successfully!\n";
        return;
    }
    Node *curr = head;
    while (curr->next != NULL)
    {
        if (curr->next->name == sname)
        {
            Node *temp = curr->next;
            curr->next = temp->next;
            if (temp == tail)
            {
                tail = curr;
            }
            delete temp;
        }
    }
}

```

```

        cout << "\nPatient removed successfully!\n";
        return;
    }
    curr = curr->next;
}
cout << "\nPatient not found in the records!\n";
}
void search(string sname)
{
    Node *curr = head;
    while (curr != NULL)
    {
        if (curr->name == sname)
        {
            cout << "\n\n*** Patient Info ***";
            curr->displayInfo();
            return;
        }
        curr = curr->next;
    }
    cout << "\nPatient Not Found!\n";
}
int main()
{
    int ch;
    do
    {
        cout << "\n=====";
        cout << "\n  Hospital Management System";
        cout << "\n    Using Linked List";
        cout << "\n=====";
        cout << "\n1. Add Patient";
        cout << "\n2. Remove Patient";
        cout << "\n3. Display All Patients";
        cout << "\n4. Search Patient";
        cout << "\n5. Exit";
        cout << "\n\nEnter your Choice: ";
        cin >> ch;
        switch (ch)

```

```

{
case 1:
    add_patient();
    break;
case 2:
    remove_patient();
    break;
case 3:
    display();
    break;
case 4:
{
    string sname;
    cout << "\nEnter the name of Patient you want to Search: ";
    cin >> sname;
    search(sname);
    break;
}
case 5:
    cout << "\nExiting program...\n";
    exit(0);
default:
    cout << "\nInvalid Choice!!\n";
}
} while (ch != 5);
return 0;
}
/*

```

Hospital Management System
Using Linked List

1. Add Patient
2. Remove Patient
3. Display All Patients
4. Search Patient
5. Exit

Enter your Choice: 1

Enter Patient Name: dia

Enter Age: 19

Enter Patient Condition: Critical

Patient added successfully!

Hospital Management System

Using Linked List

1. Add Patient

2. Remove Patient

3. Display All Patients

4. Search Patient

5. Exit

Enter your Choice: 1

Enter Patient Name: Tan

Enter Age: 20

Enter Patient Condition: Normal

Patient added successfully!

Hospital Management System

Using Linked List

1. Add Patient

2. Remove Patient

3. Display All Patients

4. Search Patient

5. Exit

Enter your Choice: 1

Enter Patient Name: Prachi

Enter Age: 32

Enter Patient Condition: Normal

Patient added successfully!

```
=====
Hospital Management System
Using Linked List
=====
```

1. Add Patient
2. Remove Patient
3. Display All Patients
4. Search Patient
5. Exit

Enter your Choice: 3

**** Patient Records ****

1) dia -> 2) Tan -> 3) Prachi -> NULL

```
=====
Hospital Management System
Using Linked List
=====
```

1. Add Patient
2. Remove Patient
3. Display All Patients
4. Search Patient
5. Exit

Enter your Choice: 2

Enter the patient name to be removed: Tan

Patient removed successfully!

```
=====
Hospital Management System
Using Linked List
```

-
-
1. Add Patient
 2. Remove Patient
 3. Display All Patients
 4. Search Patient
 5. Exit

Enter your Choice: 3

**** Patient Records ****

1) dia -> 3) Prachi -> NULL

Hospital Management System
Using Linked List

1. Add Patient
2. Remove Patient
3. Display All Patients
4. Search Patient
5. Exit

Enter your Choice: 4

Enter the name of Patient you want to Search: Prachi

*** Patient Info ***

ID: 3

Name: Prachi

Age: 32

Condition: Normal

Hospital Management System
Using Linked List

1. Add Patient

2. Remove Patient
3. Display All Patients
4. Search Patient
5. Exit

Enter your Choice: 5

Exiting program...

Process returned 0 (0x0) execution time : 88.214 s

Press any key to continue.

*/

Code Analysis Report

Hospital Management System (Singly Linked List, C++)

1. Program Purpose

The program implements a small console-based Hospital Management utility whose core educational goal is to demonstrate singly linked list operations — insertion at tail, deletion by value, traversal, and linear search — applied to a collection of patient records (ID, name, age, condition). It lets a user add patients, remove a patient by name, display all patients as a linked chain, and search for a patient by name, satisfying a typical assignment brief on linked-list-based record management.

2. Program Logic and Structure

- **Data model:** A Node class bundles patient fields (patient_id, name, age, condition) together with a next pointer, so the same class serves as both the data record and the list node. A static int cnt shared across all Node instances assigns each new patient a unique, ever-increasing ID.
- **Global list state:** head and tail are global Node* pointers maintained across the program's lifetime, letting add_patient() append in O(1) instead of walking the whole list.
- **Menu loop:** main() runs a do-while loop presenting a 5-option menu (Add, Remove, Display, Search, Exit) and dispatches on a switch(ch) statement until the user chooses Exit (case 5, which calls exit(0)).
- **Add:** add_patient() allocates a new Node, calls get_info() to read name/age/condition from the user, assigns it the next cnt value as patient_id, and links it in at tail — updating both head and tail correctly for the empty-list case.
- **Remove:** remove_patient() reads a name, handles the special case where the match is at head (re-pointing head, and clearing tail if the list becomes empty), otherwise walks the list with a trailing curr pointer to unlink the matching node and fix up tail if the removed node was the last one.
- **Display:** display() walks from head to the end, printing each patient's ID and name in an arrow-chained format (id) name -> id) name -> ... -> NULL) that visually represents the linked list structure.
- **Search:** search() takes a name, walks the list until it finds a match, and prints the full patient record via displayInfo(); if no node matches, it reports "Patient Not Found!".

3. Time Complexity Analysis

Operation	Best Case	Average Case	Worst Case	Notes
add_patient()	O(1)	O(1)	O(1)	Tail pointer makes append constant time regardless of list size
remove_patient()	O(1)	O(n)	O(n)	Best case when the match is at head; worst case when it's the last node or absent
display()	O(n)	O(n)	O(n)	Must visit every node exactly once to print it
search()	O(1)	O(n)	O(n)	Best case when the match is at head; worst case when it's the last node or absent

n denotes the number of patients currently in the list. Unlike the array-based sorting assignment, there is no comparison-count instrumentation in this program, so complexity here is derived from code inspection of loop bounds rather than a printed metric.

Overall program time complexity (dominated by remove/search in the worst case, and by display when listing all records): $O(n)$ per operation, with the menu/I/O wrapper itself contributing only $O(1)$ per iteration.

4. Space Complexity Analysis

- **Per-node storage:** Each patient consumes one heap-allocated Node containing an int ID, a string name, an int age, a string condition, and a next pointer — $O(1)$ space per patient, so the list as a whole is $O(n)$ for n patients.
- **No auxiliary copying:** Unlike algorithms that duplicate the dataset before processing, every operation here (add, remove, display, search) works directly on the live list with $O(1)$ extra space beyond the traversal pointer(s) it uses (curr, temp).
- **Static counter:** The static int cnt is a single shared $O(1)$ piece of state, not per-node, and persists for the life of the program to guarantee unique IDs even after deletions.

Overall space complexity: $O(n)$ for the list itself, $O(1)$ auxiliary space for every operation.

5. Coding Practices Observed

- Clear separation between get_info() (input) and displayInfo() (output) within Node keeps the record's own I/O self-contained.
- Correct head/tail bookkeeping: both add_patient() and remove_patient() properly handle the edge cases of an empty list and of removing the only remaining node (resetting tail to NULL).
- The unique, non-reused patient_id via a static counter is a thoughtful touch — IDs remain stable and unambiguous even as records are removed.
- display()'s arrow-chained output (id) name -> ... -> NULL) is a nice pedagogical visualization of the underlying linked list shape, not just a plain table.
- Menu-driven structure with a do-while loop and switch is easy to follow and mirrors the pattern used elsewhere in the course.
- The header comment block (assignment number, title, name, PRN, class) follows expected academic submission conventions, and a full sample run is included as a trailing comment.

6. Possible Issues and Limitations

- **Heavy reliance on bits/stdc++.h and using namespace std:** convenient for competitive-style code but considered poor practice in production or maintainable codebases; it pulls in the entire standard library and risks name collisions.

- **No input validation:** `cin >> age` does not check for non-numeric input, and `cin >> ch` (menu choice) has the same gap; a malformed entry leaves the stream in a fail state and can cause the menu loop to spin on "Invalid Choice!!" or produce garbage data instead of a graceful error message.
- **Names with spaces break input:** `cin >> name` and `cin >> sname` both stop at the first whitespace, so a two-word patient name like "Diya Bhansali" is silently truncated to "Diya", with "Bhansali" left in the buffer to corrupt the next read.
- **No duplicate-name handling:** Two patients can share a name; `remove_patient()` and `search()` both act on the first match only (by list order from head), silently ignoring any later patient with the same name.
- **No case-insensitive matching:** a search or removal for "prachi" will not match a stored record named "Prachi", since `std::string` equality is case-sensitive.
- **Search and remove are name-only:** Neither function offers lookup by the unique `patient_id`, even though the ID is the one field guaranteed to be unambiguous — this is the same category of gap as the missing name-based sort in the sorting assignment.
- **No persistence:** All records are lost when the program exits; there is no file or database backing to save state across runs.
- **No destructor / cleanup routine:** Nodes still linked when the program calls `exit(0)` or returns from `main()` are never explicitly deallocated. This is not a functional bug in a short-lived console program, but it is a leak in principle and bad practice if the code were extended into a longer-running application.
- **exit(0) inside a switch:** works but bypasses normal cleanup/RAII destructors; returning a sentinel and breaking out of the outer loop is generally considered better style than calling `exit()` directly from deep inside application logic.
- **No age or condition validation:** age is read as a raw int with no bounds check (e.g., negative or unrealistic values are accepted), and condition is a free-text string with no constraint to a fixed set of valid values (e.g., Critical/Normal/Stable).

7. Suggested Optimizations and Improvements

- Replace `cin >> name` / `cin >> sname` with `getline(cin >> ws, name)` so multi-word patient names are captured correctly.
- Validate numeric input (age, menu choice ch) with a loop that checks `cin.fail()`, calls `cin.clear()` and `cin.ignore(numeric_limits<streamsize>::max(), '\n')`, and re-prompts instead of leaving the stream corrupted.
- Add a search-by-ID and remove-by-ID option, since `patient_id` is the only field guaranteed unique — this resolves the duplicate-name ambiguity entirely.
- Wrap head, tail, and the four operations into a `PatientList` class instead of relying on global pointers, improving encapsulation and making the code easier to test and extend.
- Add a destructor (or an explicit `clear()` function called at exit) that walks the list and deletes every remaining node, to avoid relying on process teardown to reclaim memory.
- Separate the Patient (data) fields from the Node (list-plumbing) class, e.g. a Node holding a `Patient*` data pointer, to decouple patient logic from linked-list mechanics.
- Add file persistence (CSV or binary) so patient records survive across program runs, which is a natural extension for a hospital record system.
- Constrain condition to a fixed set of valid values (e.g., an enum or a validated whitelist) rather than accepting arbitrary free text.
- Consider using `std::unique_ptr<Node>` for next to get automatic memory management and eliminate manual delete calls entirely.

8. Summary

The program is a solid, well-documented academic implementation that correctly demonstrates the core singly linked list operations — constant-time append via a maintained tail pointer, linear traversal for display, and linear search/removal by value — with proper head/tail bookkeeping in every edge case, including empty-list and last-node scenarios. The sample I/O trace included in the source file confirms the program behaves as intended for well-formed input. The main opportunities for improvement lie in adding input validation and multi-word name support, offering ID-based search and removal to remove ambiguity between same-named patients, separating list mechanics from patient data for better design, and adding persistence so hospital records are not lost when the program exits.